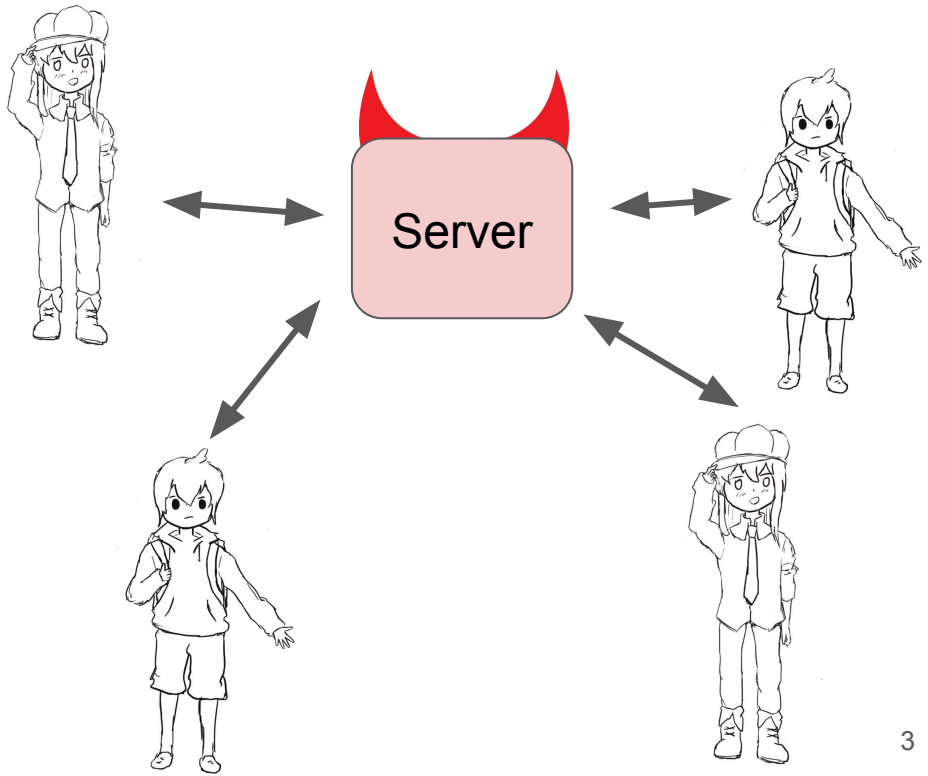
Metadata-hiding communications systems

CS 161 Spring 2026 - Lecture 32

Professor, thank you for your excellent lectures.

Last time: End-to-end encrypted messaging

Goal: Clients can exchange messages without revealing to the server what they are saying **even if the server is actively malicious**.



Key tools

- Contact discovery → Hardware enclaves
- Key lookups → Transparency logs
- Messaging → Key ratcheting

Limitation: Server still learns who you are talking to and how much you are saying.

Today: Hiding communications **metadata**

Data = What you are saying

- Email body
- SMS message contents
- HTTP response body

We have an excellent tool for hiding data: Encryption!

→ Precise security guarantees under “nice” computational assumptions

→ Minimal overhead

→ Built into real systems: TLS, Signal, etc.

Today: Hiding communications **metadata**

Metadata = Who you are talking to, when you are talking, how much you say

- Source and destination addresses — e.g., IP address, phone number, email address
- Timing information — e.g., Date/time sent
- Message length
- Packet headers

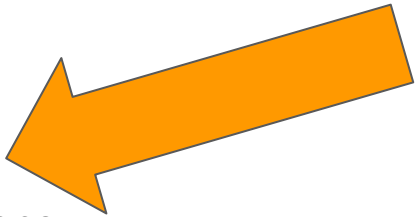
Metadata reveals: Your politics, your health conditions, your location, your favorite sports teams, who your friends are, what you like to read, how many hours a day you stream movies, how often you call your parents, etc. etc. etc.

We do not have great tools for hiding metadata from network adversaries.

→ This is an area with lots of unsolved problems, great research questions!

Hiding communications metadata: State of the art

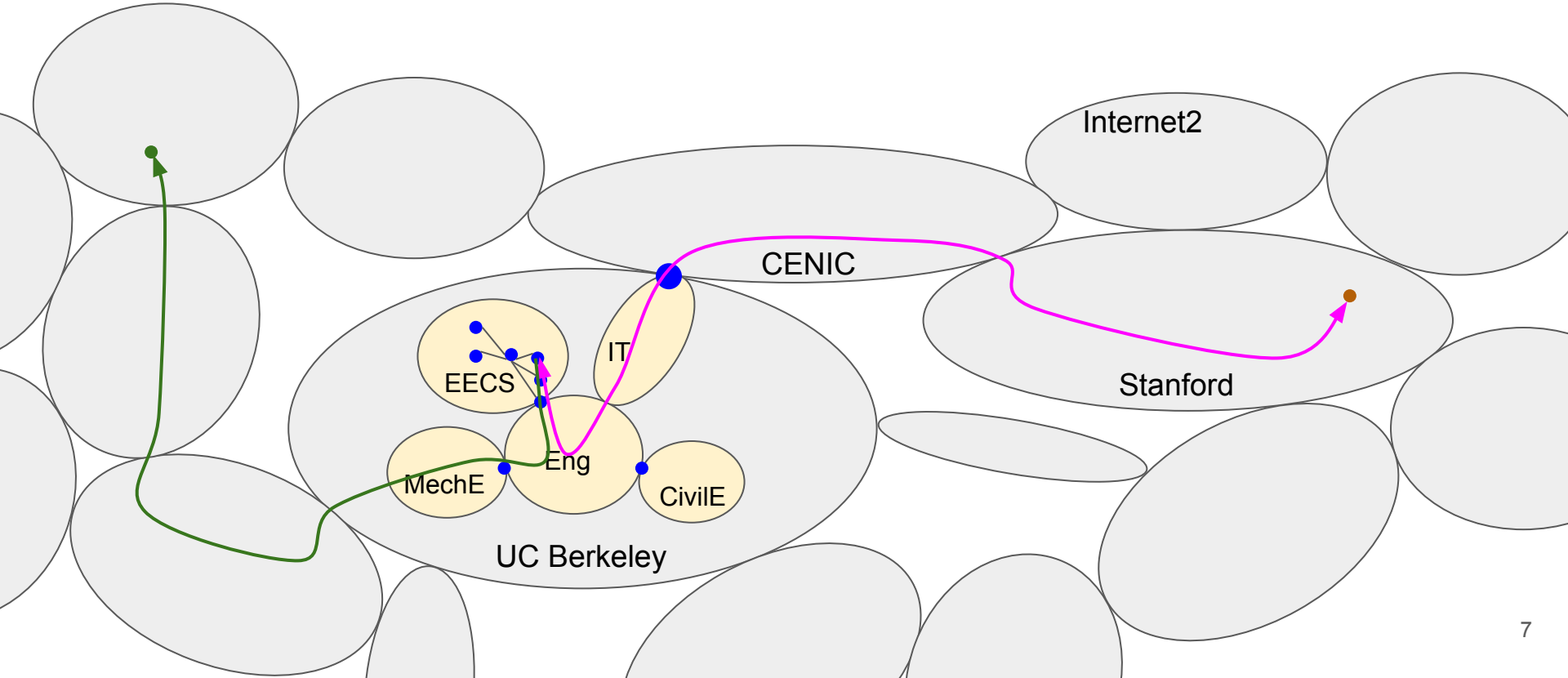
Very roughly, there are two types of systems



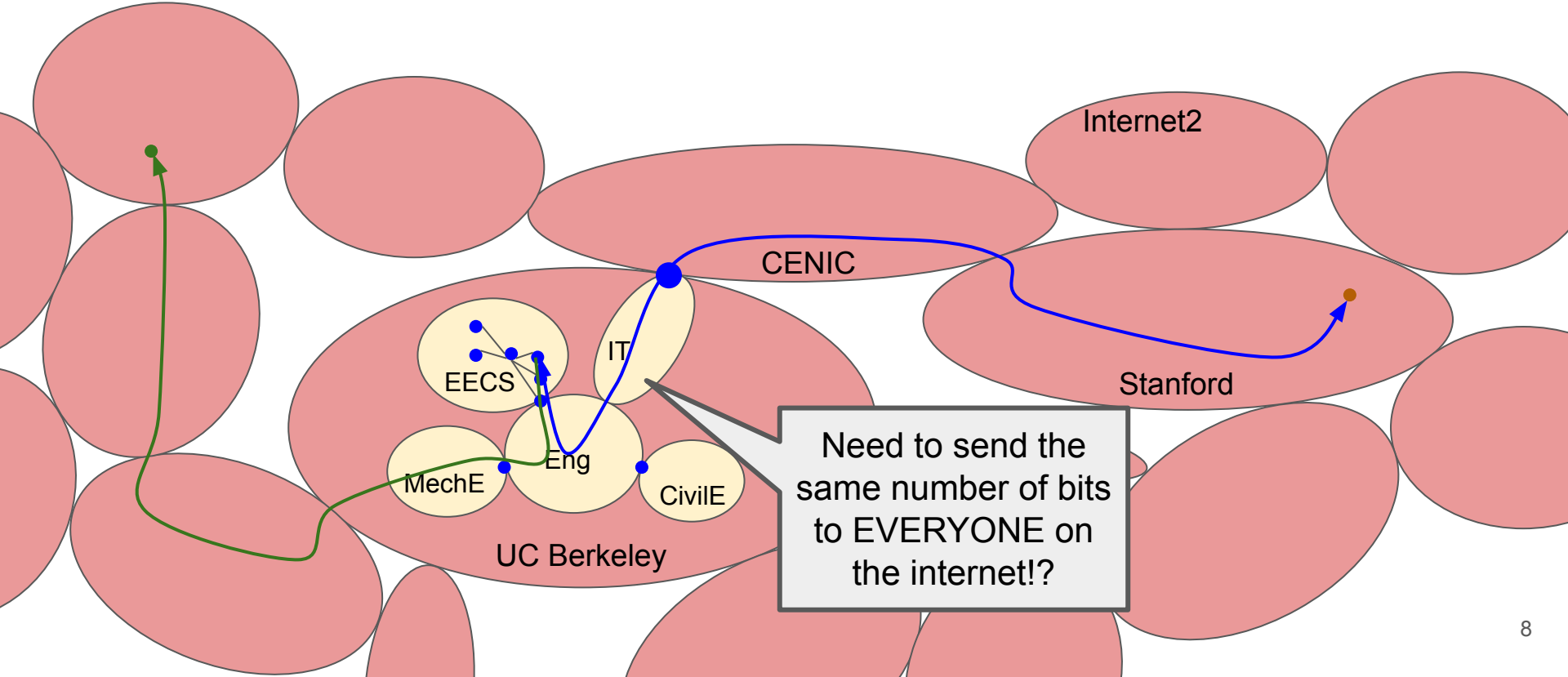
1) **Security oriented**
   - Precise security guarantees under nice cryptographic assumptions
   - Proofs that they protect metadata against certain attackers
   - Mostly confined to research papers

2) **Performance oriented**
   - “Good-enough” security
   - Focus on usability: E.g., can browse the web with some metadata protections
   - A few major open-source projects

Hiding metadata is hard
if you want to protect against a “global” network adversary



Hiding metadata is hard
if you want to protect against a “global” network adversary



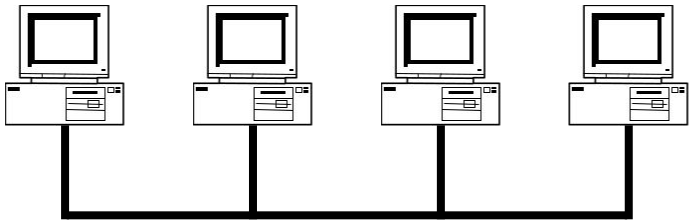
DC Nets



Hiding metadata: DC Nets (invented by Berkeley PhD student David Chaum)

Setup:

- There are *N* computers on the Internet
- The attacker controls some of them
  and can see all network traffic



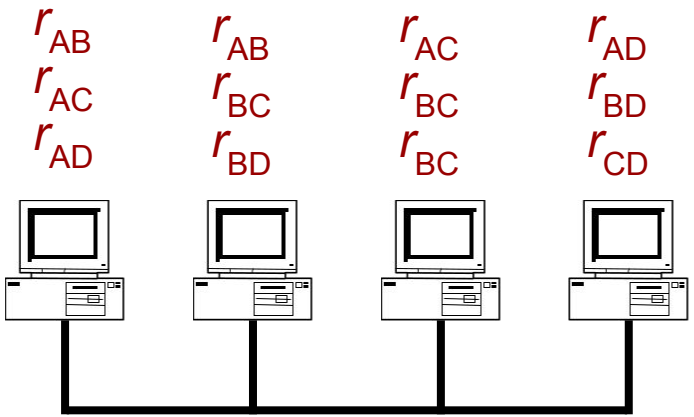
Goal: "Anonymous broadcast"

- Each client should be able to publish messages that everyone can see
- The attacker should learn no information about who published what
  … beyond what the messages themselves reveal

Hiding metadata: DC Nets (invented by Berkeley PhD student David Chaum)

More setup:

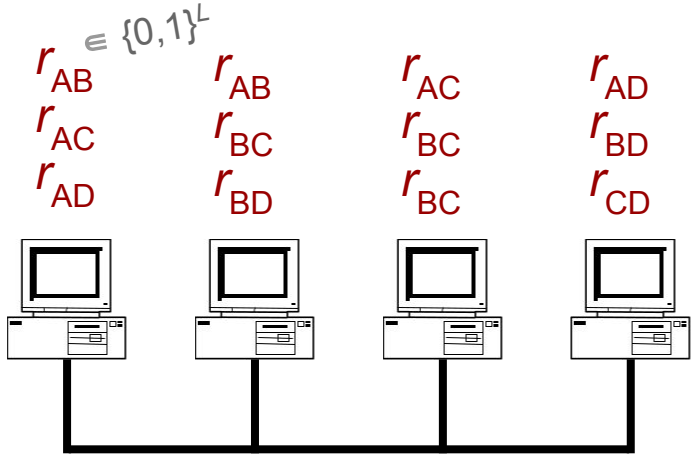
- DC nets operates in *rounds*
- In each round, every node either:
  - Anonymously broadcasts an *L*-bit message, or
  - Stays silent
- We assume that every pair of parties has an *L*-bit shared secret per round
  - Can generate with DH key exchange + pseudorandom function



Hiding metadata: DC Nets (invented by Berkeley PhD student David Chaum)

Scheme:

- In each round, every party computes the XOR of their shared secrets
- If someone wants to transmit, they XOR in their message



Alice sends $r_{AB} \oplus r_{AC} \oplus r_{AD}$

Bob sends $r_{AB} \oplus r_{BC} \oplus r_{BD} \oplus \mathbf{msg}$

Carol sends $r_{AC} \oplus r_{BC} \oplus r_{CD}$

Dave sends $r_{AD} \oplus r_{BD} \oplus r_{CD}$

Collect messages from all clients and XOR them together to get output

DC Nets: Properties

“Best possible” security

- Assume that attacker controls entire network and any subset of clients
- Even then, the attacker can’t tell which honest client sent the message!
  - The IP headers don’t reveal anything about who is sending what

Problems:

- Poor use of network.
  For every bit of output, each of the *N* players must send 1 bit ⇒ 1/*N* goodput
- Not very robust.
  One malicious client can *anonymously* jam the channel

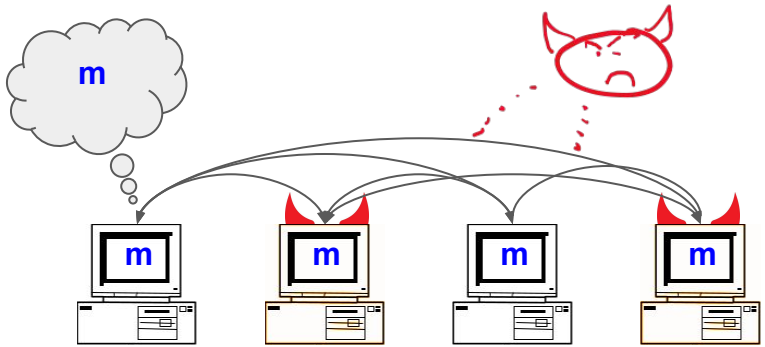


CoverDrop



In some applications, a weaker security guarantee suffices

Anonymous broadcast



Metadata-private drop-box

CoverDrop

- Source wants to contact journalist at newspaper
- Source doesn't want to start a Signal chat with reporter
  - Leaks to the network the fact that the source and reporter are talking

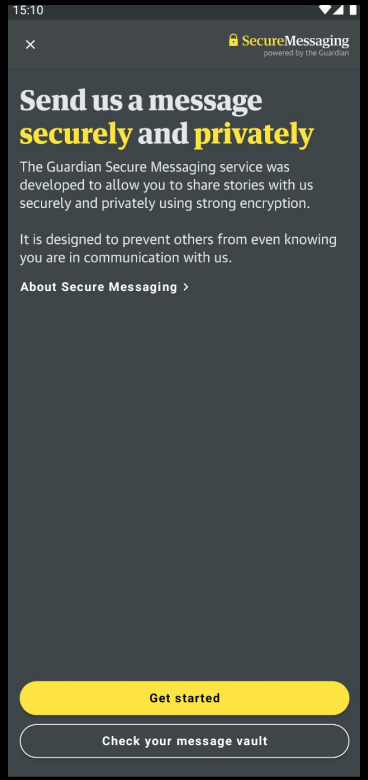
Idea:

- Many newspapers have mobile apps
- Have all users of mobile app send a fixed-length message to journalist every hour
  - Most users send: Enc($PK_{newspaper}$, "0000000") // "Cover traffic"
  - Source sends: Enc($PK_{newspaper}$, `secret_source_material`)
- Attacker seeing the entire network can't distinguish:
  - Millions of casual newspaper readers from
  - Secret source

→ Simple design
→ Strong, precise security guarantee

Actually used in production!

The Guardian



CoverDrop White Paper

Daniel Hugenroth[2], Sam Cutler[1], Dominic Kendrick[1], Mario Savarese[1], Zeke Hunter-Green[1], Philip McMahon[1], Marjan Kalanaki[1], Diana A. Vasile[2], Sabina Bejasa-Dimmock[1], Luke Hoyland[1], and Alastair R. Beresford[2]

[1]The Guardian
[2]University of Cambridge

The free press fulfills an important function in a democracy. It can provide individuals with a mechanism through which they can hold powerful people and organizations to account. In previous work, the University of Cambridge developed CoverDrop: a set of extensions to a typical news app which provided a secure and usable method of establishing initial contact between journalists and sources. Since publication, The Guardian and the University of Cambridge have undertaken further work on the design, highlighting additional challenges and shortcomings which needed to be addressed before deployment. This white paper presents an updated design of the CoverDrop system which addresses these issues, and describes the version that The Guardian first deployed in April 2025. You can find this paper and more information at `https://www.coverdrop.org`.

1 Introduction

The Snowden leaks revealed that nation states develop and operate extensive mass-surveillance infrastructure [1]. These capacities have potential for abuse. They could for example be deployed against news organizations and potential sources who are attempting to expose wrongdoing within governments or other powerful organizations. As a consequence, significant care is required to ensure that potential sources are not exposed. However, many leading news organizations today suggest that sources use initial methods of contact that are insecure or are hard to use. Such methods can, for example, fail to adequately protect either the confidentiality of the communication itself, or have the potential to expose the relationship between a source and a journalist to a network observer capable of performing traffic analysis. In prior work, we conducted two workshops with British news organizations and surveyed source communication

Our central insight towards a practical solution is that news organizations already run a widely-available platform from which they can offer a secure, usable method of initial contact: the news app on a smartphone. The confidentiality and integrity of message content can be assured through widely-available cryptography on these platforms; journalists can be authenticated directly by the news organization; traffic analysis by a network operator or state actor can be thwarted by requiring all the installations of the news app to produce cover traffic, thus hiding whether any given user is in contact with a journalist or not; the system does not require users to install specialist software or tools; and the news app can provide a usable interface which is similar in style and operation to a typical messaging app.

In previous work, we described CoverDrop, a set of extensions to a typical news app which provides a secure and usable method of initial contact between journalists and sources [2]. This white paper presents a revised version of CoverDrop that The Guardian deployed for the first time in 2025. We describe the planned system, implementation details (§2, §4–§6, §8–§10), the requirement analysis (§3) that motivates our design decisions, and discuss the user interface (§7).

To make the document approachable for readers from a variety of backgrounds, we highlight the main areas of sections. GENERAL sections provide important overview of the system and its environment. The cryptographic details and anonymity guarantees are highlighted with PROTOCOL. Since the usability of the system is critical, several sections focus on UX. Software engineering best-practices and specific deployment considerations are marked with SWE.

Our implementation is available as an open-source repository under an Apache 2.0 license: `https://gith`

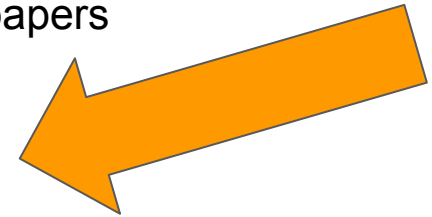
Question:

Are there other settings in which we can get these strong metadata protections at modest cost?

Hiding communications metadata: State of the art

Very roughly, there are two types of systems

1) **Security oriented**
   - Precise security guarantees under nice cryptographic assumptions
   - Proofs that they protect metadata against certain attackers
   - Mostly confined to research papers



2) **Performance oriented**
   - “Good-enough” security
   - Focus on usability: E.g., can browse the web with some metadata protections
   - A few major open-source projects

Web browsing with metadata protection

- When you browse the web, network attacker learns what sites you're visiting
- It is difficult to protect web metadata privacy against a network adversary
  - E.g., streaming a 4 GB movie will look different from downloading a 1 KB web page
  - Unless you make everyone who wants a 1 KB page download 4 GB of data

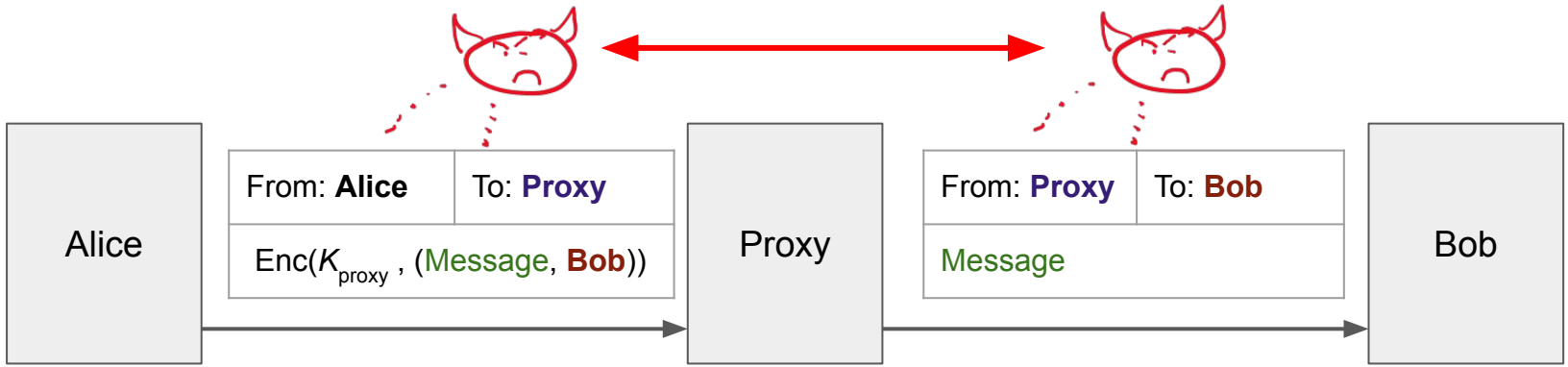
We can defend against localized network attackers

Three techniques, in increasing order of strength:

- One-hop proxy (e.g., VPN)
- Apple's iCloud Private Relay
- Tor

One-hop proxy

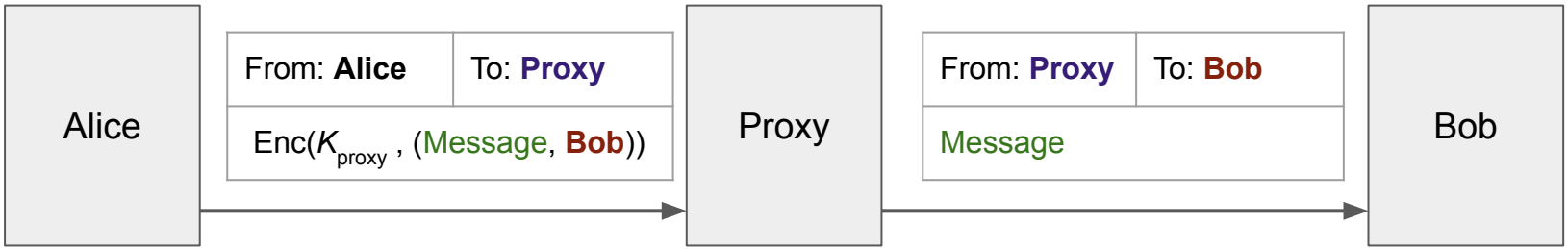
Proxy: A third party that relays Internet traffic



- Provides **some protection** against a localized network attacker
  - Attacker does not see destination IP in the clear
  - Attacker can still inspect volume/pattern of packets to learn sensitive info
- Provides **no protection** against a malicious proxy

One-hop proxy

Proxy: A third party that relays Internet traffic

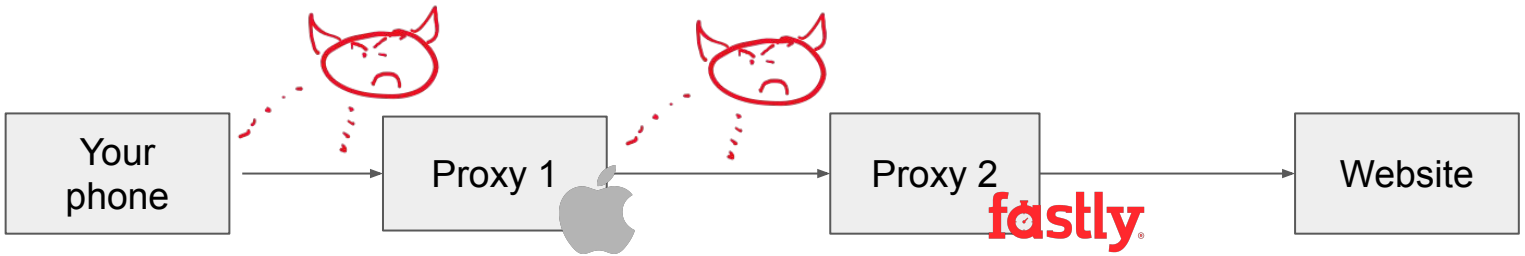


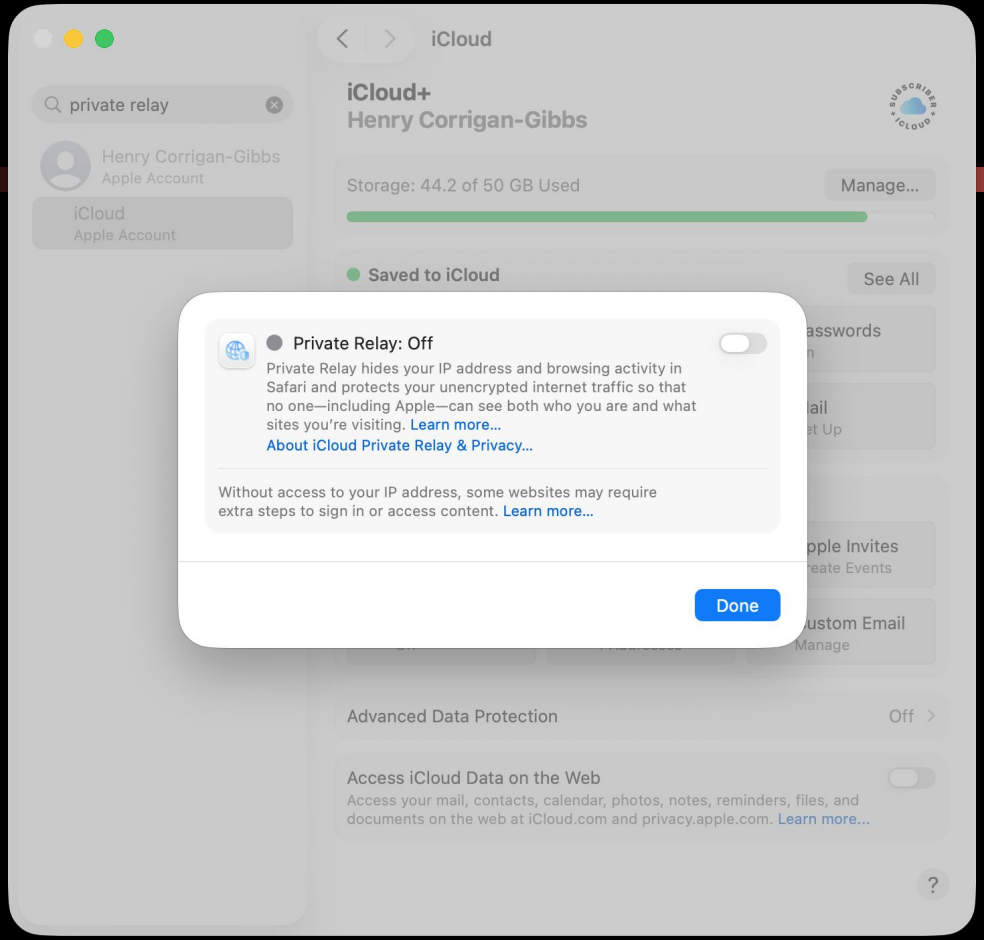
Many implementations:

- Oblivious HTTP – At application layer (HTTP proxy)
- SOCKS proxy – At transport layer (TCP proxy)
- VPN – At Internet layer (IP proxy)

Two-hop proxy: Apple's iCloud Private Relay

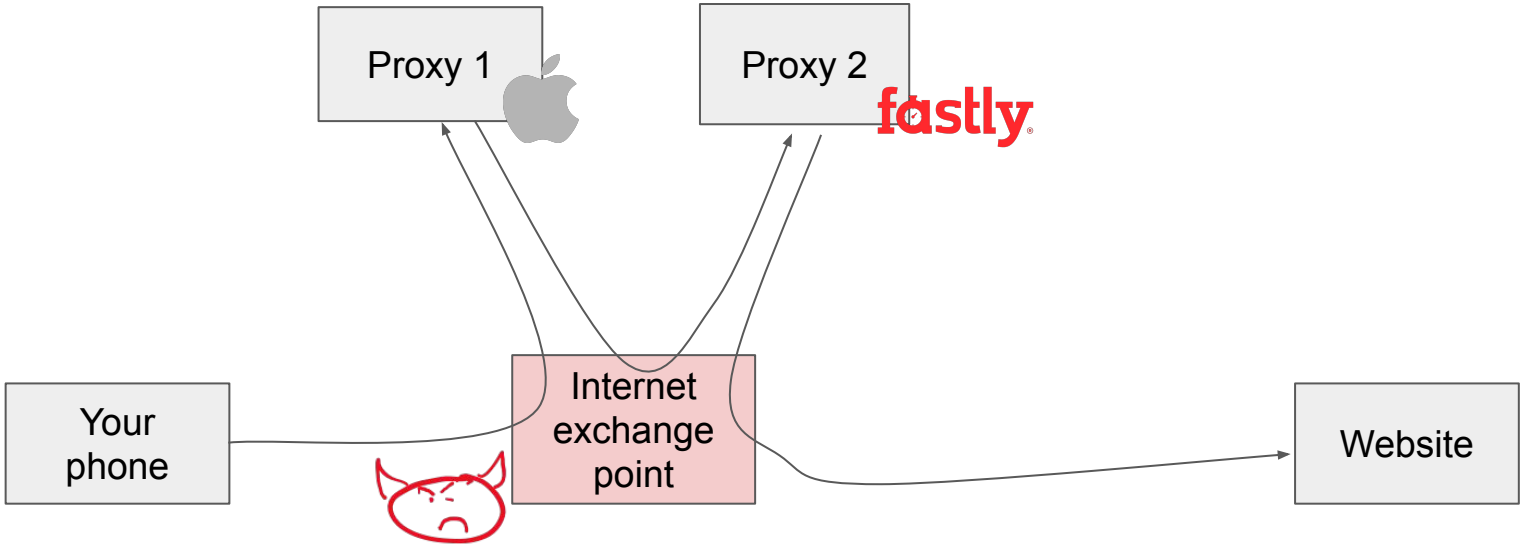
- To protect against the compromise of the proxy server, add more proxies!
  - Traffic to Proxy 2 is proxied through Proxy 1
  - Traffic to website is proxied through Proxy 2
- Apple's Private Relay system does this
  - Independent companies (CloudFlare, Akamai, Fastly) run the "exit" proxy
  - Provides some protection against proxy compromise
  - Still does not protect against traffic-analysis attacks




iCloud
private relay
Henry Corrigan-Gibbs
Apple Account
iCloud
Apple Account
iCloud+
Henry Corrigan-Gibbs
Storage: 44.2 of 50 GB Used
Manage...
Saved to iCloud
See All
Private Relay: Off
Private Relay hides your IP address and browsing activity in Safari and protects your unencrypted internet traffic so that no one—including Apple—can see both who you are and what sites you're visiting. Learn more...
About iCloud Private Relay & Privacy...
Without access to your IP address, some websites may require extra steps to sign in or access content. Learn more...
Done
Advanced Data Protection
Off
Access iCloud Data on the Web
Access your mail, contacts, calendar, photos, notes, reminders, files, and documents on the web at iCloud.com and privacy.apple.com. Learn more...

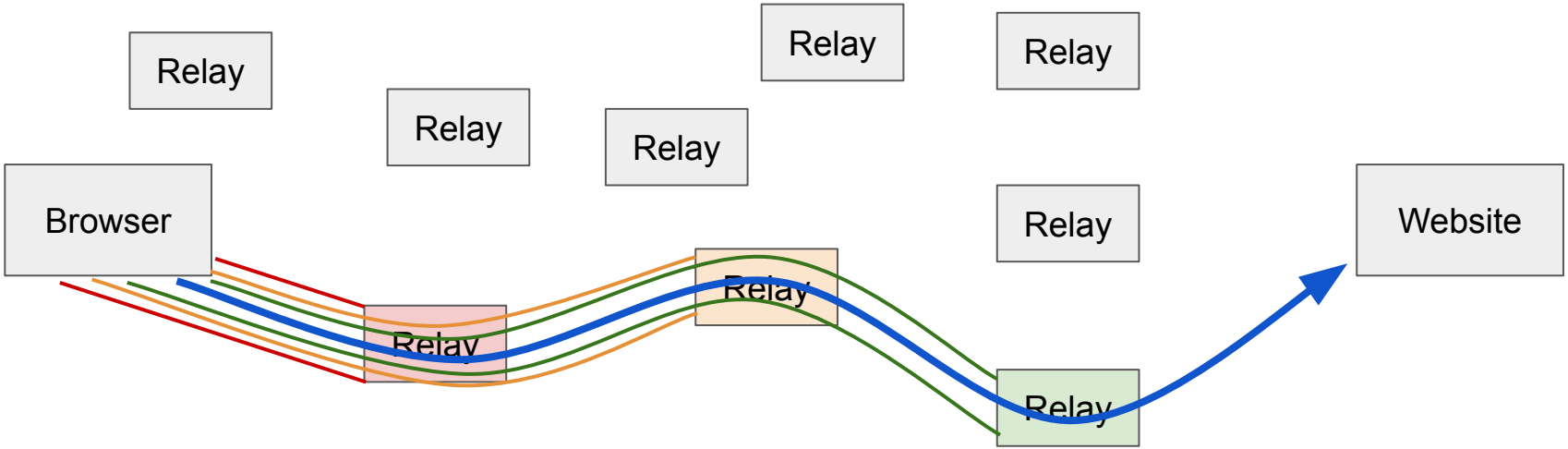
Two-hop proxy: Apple's iCloud Private Relay

- It's not clear how difficult it is to pull of traffic-analysis attacks
- Attacker at a privileged position in the network can learn a lot

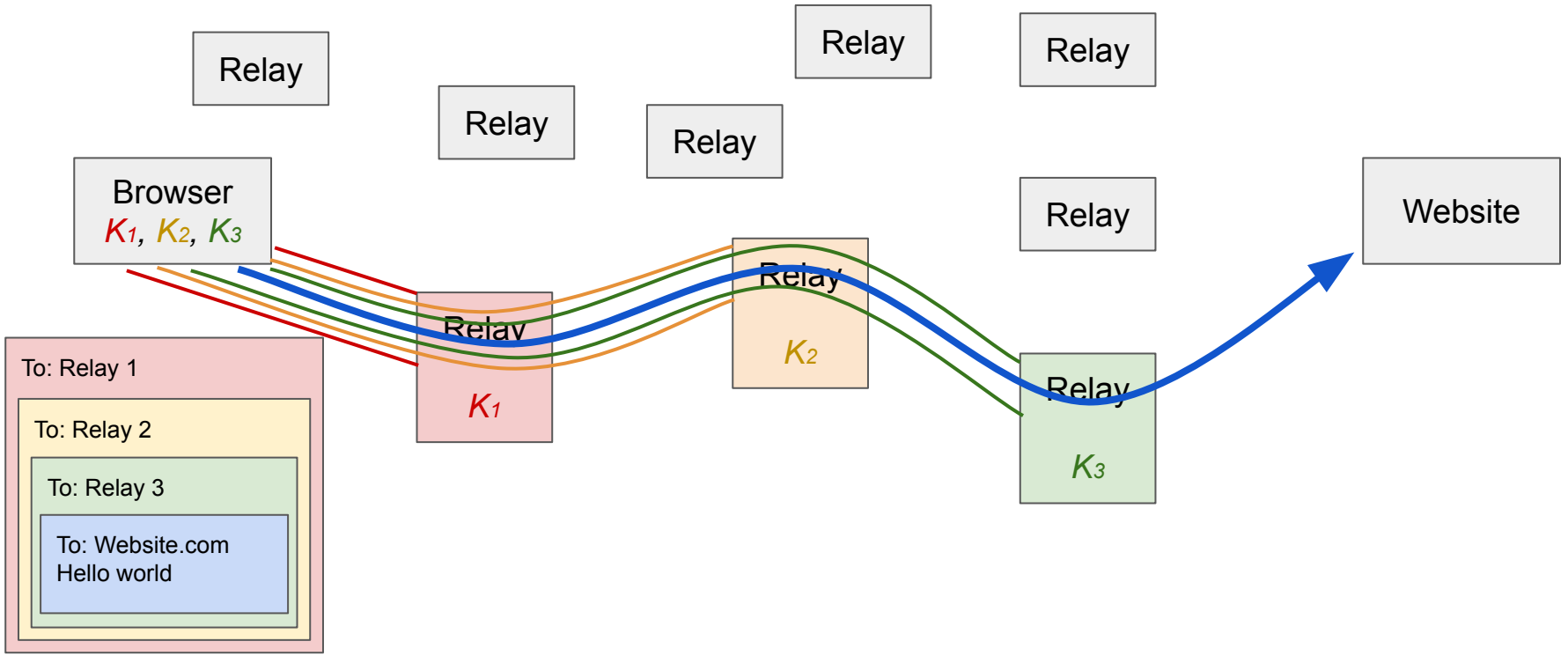


Three-hop proxy: Tor

- Tor is an open-source project that maintains thousands of “relay” servers
- Tor browser picks a three-hop path through this network, sends traffic through
- Seems to work in practice?? Hard to know!



Three-hop proxy: Tor



Summary: Metadata-hiding communication

Strong, cryptographic metadata protection is sometimes possible

- Example: CoverDrop
- Usually works in low-bandwidth applications
- Often requires sending lots of cover traffic

For web browsing, we usually give up on getting precise security guarantees

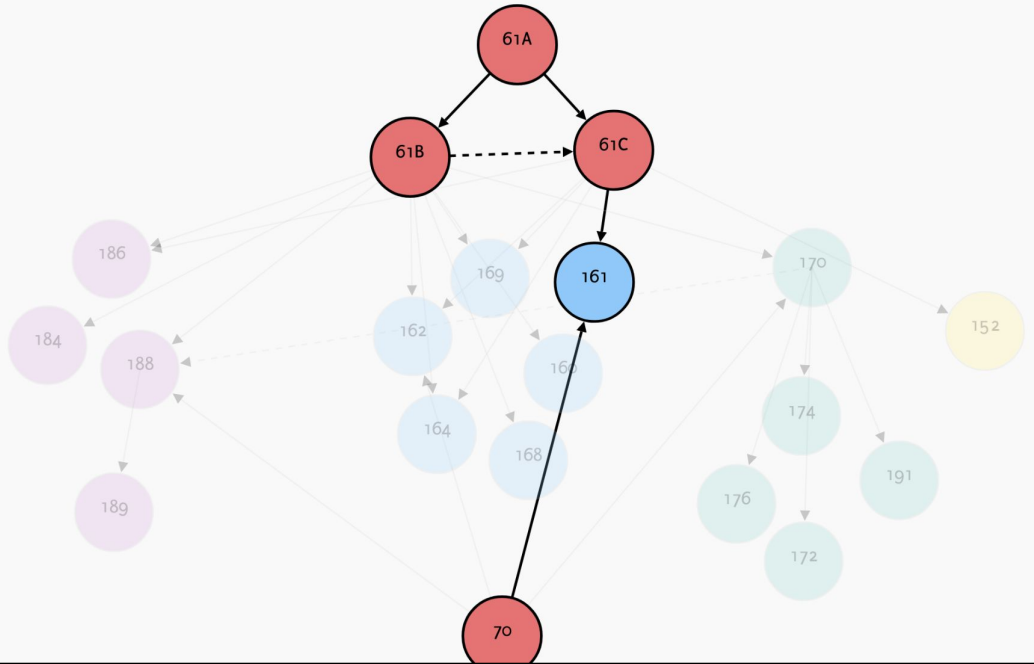
- “Plug and pray” = Use Private Relay/Tor, hope your adversary is not too smart
- This is unsatisfying
- Maybe YOU will come up with a better solution!

Closing thoughts

Is this the end of the line!?

COURSE GUIDES

We offer a comprehensive course guide for all undergraduate level EECS courses here.



Courses you can take

- CS 171: **Cryptography** (taught Fall 2026)
- CS 261: **Grad computer security**, undergrads allowed with permission
- CS 276: **Grad cryptography**
- CS 294: **Special topics** classes
  - I'll be teaching one in Spring 2027
  - Likely on metadata-hiding communications systems

Other on-campus opportunities

- Bay Area Crypto Day
  - At Berkeley ~1x per year
- Berkeley Security Seminar
  - Once per week
- Berkeley Cryptography Seminar
  - From time to time
- Bay Area Security Day
  - Likely starting Fall 2026

Final thoughts

- **It has been an absolute joy to be your instructor this semester**
  - If it wasn't obvious, I love this material and I love teaching
  - I hope that this enthusiasm is a little bit contagious!

- **I hope that this class has changed the way you think**, at least a little bit
  - About computers, about the Internet, about security, …

- **And I hope that in future years, YOU will change the way the world thinks**
  - CS161 students have had tremendous impact on the industry, on research, etc.

THE END.